DOCUMENTO MAESTRO DE DESARROLLO

PLATAFORMA ENTERPRISE DE\nREDES NEURONALES ARTIFICIALES

Aplicación práctica de las operaciones de la Semana 11

React + TypeScript + API Python + TensorFlow/Keras + PostgreSQL

ESPECIFICACIÓN FUNCIONAL, TÉCNICA Y ARQUITECTÓNICA\nVersión 1.0 | Octubre 2026

1. Resumen ejecutivo

Se propone construir una plataforma web empresarial para diseñar, entrenar, evaluar, versionar y ejecutar modelos de redes neuronales artificiales. El sistema transforma las operaciones de la Semana 11 en un flujo práctico: comprender la red neuronal, definir sus capas y conexiones, reconocer familias de redes y crear un modelo con TensorFlow/Keras. El contenido curricular identifica expresamente neuronas, capas, conexiones, funciones de activación, redes Feedforward, Recurrentes, Convolucionales y GAN, además de la creación de redes con TensorFlow y Keras. filecite turn0file0 L270-L293

La solución se plantea como un producto real, modular y extensible, no como una página demostrativa aislada. React será la interfaz de trabajo; una API en Python ejecutará las tareas de aprendizaje automático; TensorFlow/Keras gestionará los modelos; PostgreSQL almacenará usuarios, proyectos, configuraciones, ejecuciones y métricas; y un canal de eventos permitirá observar el progreso de los trabajos en tiempo real.

2. Objetivo del sistema

Desarrollar una plataforma enterprise que permita a usuarios autorizados crear proyectos de IA, configurar arquitecturas de redes neuronales, ejecutar entrenamientos, consultar métricas y comparar versiones desde una interfaz React responsive, con trazabilidad, seguridad y procesamiento asíncrono.

2.1 Objetivos específicos

- Explicar visualmente los componentes de una red neuronal: neuronas, capas, conexiones, pesos y funciones de activación.
- Permitir seleccionar una familia de red y configurar una arquitectura compatible.
- Crear, validar, entrenar y evaluar modelos usando TensorFlow/Keras desde el backend Python.
- Mostrar estado, porcentaje de avance, época actual, pérdida y métricas sin recargar la página.
- Guardar proyectos, configuraciones, historial de ejecuciones y versiones de modelos.
- Aplicar autenticación, permisos por rol, auditoría, validación de entradas y control de errores.

3. Alcance académico: operaciones de la Semana 11

La fuente curricular sitúa en la Semana 11 la tarea “Define la estructura y crea una red neuronal artificial”. Sus operaciones de aprendizaje son: definir la red neuronal y su importancia en IA; describir su estructura; identificar tipos de redes; y crear una red neuronal con TensorFlow y Keras. filecite turn0file0 L270-L293

Operación / contenido	Funcionalidad del sistema	Evidencia verificable
Definir red neuronal e importancia	Módulo de fundamentos con conceptos, glosario y ejemplos.	Ficha conceptual consultable.
Describir estructura: neuronas, capas, conexiones y activaciones	Constructor visual de arquitectura con validación de capas.	Diagrama y configuración guardada.
Identificar tipos: Feedforward, RNN, CNN y GAN	Catálogo de arquitecturas con casos de uso y restricciones.	Tipo seleccionado y justificación registrada.
Crear red con TensorFlow/Keras	Asistente de creación, compilación y ejecución en API Python.	Modelo entrenado o error técnico trazable.

4. Propuesta de producto

4.1 Nombre de trabajo

NeuroLab Enterprise — Plataforma de experimentación y gestión de redes neuronales.

4.2 Usuarios y roles

- Administrador: gestiona usuarios, roles, límites de recursos y configuración global.
- Responsable de proyecto: crea proyectos, asigna integrantes y administra modelos.
- Ingeniero/analista de IA: configura arquitecturas, entrena y evalúa modelos.
- Revisor: consulta resultados, compara ejecuciones y aprueba versiones.
- Lector: acceso de solo lectura a proyectos y reportes autorizados.

4.3 Flujo principal

1. Crear proyecto y describir el problema que se desea resolver.
2. Seleccionar familia de red: Feedforward, RNN, CNN o GAN.
3. Definir entradas, capas, unidades, activaciones y salida.
4. Seleccionar dataset permitido y parámetros de entrenamiento.
5. Validar configuración y enviar el trabajo a la cola de ejecución.
6. Observar progreso en tiempo real y consultar métricas.
7. Evaluar, guardar versión, comparar experimentos y exportar informe.

5. Arquitectura empresarial

Se recomienda una arquitectura modular por dominios, con frontend y backend separados. No ejecutar TensorFlow directamente en el navegador para entrenamientos reales: el navegador debe administrar la experiencia y consumir una API; el backend Python ejecuta el trabajo en procesos controlados.

Capa	Responsabilidad y tecnología propuesta
Presentación	React + Vite + TypeScript; diseño responsive; formularios; gráficos; constructor visual; rutas protegidas.
Estado y acceso a datos	TanStack Query para consultas/cache; cliente API tipado; validación de formularios; manejo de estados de carga y error.
API de aplicación	Python + FastAPI; REST versionado (/api/v1); autorización, validación, límites y contratos OpenAPI.
Procesamiento asíncrono	Cola de trabajos y workers separados para entrenamiento; reintentos controlados; cancelación cooperativa.
Motor ML	TensorFlow/Keras; construcción de modelos desde configuraciones validadas; entrenamiento, evaluación y serialización.
Persistencia	PostgreSQL para metadatos y resultados resumidos; almacenamiento de objetos para datasets/modelos; no guardar archivos grandes como filas ordinarias.
Tiempo real	WebSocket o Server-Sent Events para estado, época, métricas y eventos; REST continúa siendo fuente de verdad.
Operación	Docker; variables de entorno; logs estructurados; métricas; copias de seguridad; CI/CD y separación dev/test/prod.

5.1 Diagrama lógico

USUARIO → REACT (UI) → API FASTAPI → COLA DE TRABAJOS → WORKER PYTHON → TENSORFLOW/KERAS\n ↕ eventos en tiempo real ↓\n **WEBSOCKET/SSE ← API ← POSTGRESQL + ALMACENAMIENTO DE OBJETOS**

El navegador recibe actualizaciones de estado, pero la ejecución continúa en el worker aunque el usuario cambie de pantalla. El servidor debe comprobar la propiedad del proyecto y los permisos en cada consulta y acción.

6. Módulos y submódulos

M01. Inicio / Dashboard

- Resumen de proyectos
- Entrenamientos activos y recientes
- Indicadores de uso y alertas

M02. Proyectos de IA

- Listado, búsqueda y filtros
- Crear/editar proyecto
- Miembros, permisos y estado
- Documentación y etiquetas

M03. Fundamentos neuronales

- Conceptos de red neuronal
- Neuronas, capas y conexiones
- Pesos, sesgos y funciones de activación
- Glosario y ejemplos guiados

M04. Diseñador de arquitecturas

- Selector de familia: Feedforward, RNN, CNN, GAN
- Editor de capas y parámetros
- Validación de formas de entrada/salida
- Vista previa del grafo y resumen de parámetros

M05. Datasets

- Registro de dataset y metadatos
- Carga controlada y validación
- Separación entrenamiento/validación/prueba
- Versionado, permisos y eliminación segura

M06. Entrenamientos

- Configuración de épocas, batch size y optimizador
- Cola y prioridad por permisos
- Inicio, cancelación y reintento
- Progreso en tiempo real

M07. Evaluación y métricas

- Pérdida y métricas por época
- Evaluación final
- Gráficos y comparación de ejecuciones
- Registro de limitaciones y observaciones

M08. Registro de modelos

- Versiones inmutables
- Metadatos de entrenamiento
- Estado borrador/revisado/aprobado/retirado
- Descarga/exportación autorizada

M09. Reportes

- Ficha de proyecto
- Resumen de arquitectura
- Historial de entrenamiento
- Exportación CSV/PDF según permisos

M10. Administración y seguridad

- Usuarios, roles y permisos
- Auditoría de acciones
- Límites de recursos
- Configuración, salud del sistema y retención

7. Diseño funcional de pantallas

- Shell empresarial: menú lateral compacto, encabezado con contexto del proyecto, breadcrumbs y perfil.
- Dashboard: tarjetas de resumen discretas, tabla de trabajos recientes y panel de estado del sistema.
- Proyecto: pestañas internas Contexto, Arquitectura, Dataset, Entrenamientos, Modelos e Historial.
- Diseñador: panel de capas, lienzo central y panel de propiedades; evitar formularios excesivamente largos.
- Entrenamiento: estado visible (En cola, Preparando, Ejecutando, Completado, Fallido, Cancelado), progreso, métricas y logs recientes.
- Historial: tabla con fecha, versión, duración, estado, parámetros principales y acciones autorizadas.
- Accesibilidad: navegación por teclado, contraste suficiente, etiquetas explícitas y estados no dependientes solo del color.

8. Modelo de datos inicial

Entidad	Campos esenciales	Relaciones / propósito
users	id, email, display_name, status	Identidad de aplicación; vincular con proveedor de autenticación.
projects	id, name, description, owner_id, status, created_at	Pertenece a un usuario propietario.
project_members	project_id, user_id, role	Acceso colaborativo por proyecto.
datasets	id, project_id, name, version, storage_key, metadata_json	Metadatos; los archivos viven en almacenamiento de objetos.
architectures	id, project_id, name, family, config_json, version	Configuración validada de capas y parámetros.
training_jobs	id, project_id, architecture_id, dataset_id, status, progress, started_at, ended_at	Estado y ciclo de vida del trabajo.
training_metrics	id, job_id, epoch, metric_name, metric_value, recorded_at	Métricas temporales por época.
model_versions	id, project_id, job_id, version, storage_key, status, checksum	Artefactos inmutables y trazables.
audit_events	id, actor_id, project_id, action, metadata_json, created_at	Trazabilidad de acciones relevantes.

9. API propuesta

Endpoint (propuesta)	Uso
GET /api/v1/projects	Listar proyectos accesibles al usuario.
POST /api/v1/projects	Crear proyecto.
GET /api/v1/projects/{id}	Consultar detalle autorizado.
POST /api/v1/projects/{id}/architectures	Crear una arquitectura validada.

POST /api/v1/projects/{id}/training-jobs	Enviar entrenamiento a cola; devolver job_id.
GET /api/v1/training-jobs/{id}	Consultar estado persistido del trabajo.
POST /api/v1/training-jobs/{id}/cancel	Solicitar cancelación si el estado lo permite.
GET /api/v1/training-jobs/{id}/events	Canal SSE o WebSocket autenticado para eventos.
GET /api/v1/projects/{id}/models	Listar versiones accesibles.
GET /api/v1/projects/{id}/audit-events	Consultar auditoría según rol.

10. Reglas de negocio y validaciones

- No iniciar un entrenamiento sin proyecto, arquitectura y dataset válidos.
- La configuración de arquitectura se valida en backend; el frontend no es una frontera de seguridad.
- Un trabajo sigue una máquina de estados explícita: QUEUED → PREPARING → RUNNING → COMPLETED / FAILED / CANCELLED.
- La cancelación es una solicitud; el worker confirma el estado final y registra el motivo.
- Cada trabajo registra versiones de código/configuración, fecha, parámetros, semilla cuando aplique y referencia al dataset.
- Los archivos se validan por tipo, tamaño, permisos y contenido; no ejecutar archivos subidos por el usuario.
- Los límites de CPU, memoria, GPU, duración y concurrencia deben ser configurables.
- Los modelos publicados no se sobrescriben: se crea una nueva versión y se conserva la trazabilidad.

11. Seguridad, privacidad y confiabilidad

- Autenticación mediante proveedor seguro; autorización RBAC y comprobación de pertenencia a proyecto.
- Validación de entrada con esquemas; protección contra acceso horizontal a recursos ajenos.
- Secretos solo en variables seguras del servidor, nunca en el bundle de React.
- Límites de tasa, tamaño de carga, duración de trabajo y cuota por usuario/proyecto.
- Auditoría de inicio/cancelación de trabajos, cambios de permisos, exportaciones y cambios de estado de modelos.
- Cifrado en tránsito, copias de seguridad probadas y política de retención.
- Aislamiento de workers; no ejecutar código arbitrario ni notebooks aportados por usuarios en el proceso principal.
- Logs sin contraseñas, tokens ni datos sensibles del dataset.

12. Requisitos no funcionales y aceptación

Área	Criterio de aceptación propuesto
Usabilidad	Un usuario autorizado crea un proyecto y configura una arquitectura mediante pasos guiados.
Tiempo real	El estado y las métricas aparecen sin recargar la pantalla; reconectar recupera el estado desde API.
Trazabilidad	Cada ejecución se consulta después de cerrar sesión y volver a ingresar.
Seguridad	Un usuario no puede leer ni cancelar trabajos de proyectos ajenos.
Confiabilidad	Los errores de entrenamiento muestran mensaje controlado y conservan logs técnicos protegidos.
Mantenibilidad	Frontend, API, workers y persistencia tienen límites de responsabilidad claros.
Pruebas	Pruebas unitarias de validación, integración API/DB y flujo end-to-end principal.

13. Estructura de carpetas recomendada

```
neurolab-enterprise/
├── apps/
│   └── web/
│       └── src/
│           ├── app/              # router, providers, configuración
│           ├── components/       # UI reutilizable y layout
│           ├── features/
│           │   ├── dashboard/
│           │   ├── projects/
│           │   ├── fundamentals/
│           │   ├── architectures/
│           │   ├── datasets/
│           │   ├── training/
│           │   ├── evaluation/
│           │   ├── model-registry/
│           │   ├── reports/
│           │   └── administration/
│           ├── lib/              # cliente API, utilidades
│           ├── hooks/
│           ├── types/
│           └── styles/
├── services/
│   └── api/
│       ├── app/
│       │   ├── api/v1/
│       │   ├── core/             # config, auth, logging
│       │   ├── modules/          # dominios y casos de uso
│       │   ├── schemas/
│       │   ├── repositories/
│       │   └── main.py
│       └── tests/
├── workers/
│   └── training/                 # tareas TensorFlow/Keras aisladas
├── infra/
│   ├── docker/
│   ├── migrations/
│   └── monitoring/
├── docs/
│   ├── architecture/
│   ├── api/
│   └── operations/
└── README.md
```

14. Plan de implementación por fases

Fase	Entregables	Condición de salida
0. Descubrimiento y diseño	Historias, roles, diagramas, criterios y prototipo UI.	Alcance aprobado y flujos definidos.
1. Base de plataforma	React shell, autenticación, API, PostgreSQL, migraciones y CI.	Login, permisos y CRUD de proyectos probados.
2. Arquitecturas	Fundamentos, selector de familia y constructor validado.	Arquitecturas válidas guardadas y versionadas.
3. Entrenamiento	Cola, worker, TensorFlow/Keras y estados persistidos.	Un modelo de prueba se entrena de extremo a extremo.
4. Tiempo real y evaluación	Eventos, métricas, gráficas, cancelación y recuperación.	UI refleja progreso y recupera estado tras reconexión.
5. Model registry y reportes	Versiones, comparaciones, exportaciones y auditoría.	Flujo de revisión y exportación probado.
6. Endurecimiento y despliegue	Pruebas de seguridad, monitoreo, backups y documentación.	Checklist de producción aprobado.

15. Estrategia de pruebas

- Unitarias: validación de arquitectura, transiciones de estado, permisos y cálculo de métricas.
- Integración: API con PostgreSQL, almacenamiento y cola de trabajos.
- Contrato: esquemas de API y eventos en tiempo real.
- End-to-end: crear proyecto → diseñar red → iniciar entrenamiento → ver métricas → guardar versión.
- Seguridad: acceso a proyectos ajenos, carga de archivos inválidos, abuso de endpoints y límites de cuota.
- Recuperación: desconexión de navegador, reinicio de API y fallo de worker sin perder el estado persistido.

16. Riesgos y decisiones técnicas

- Los entrenamientos pueden consumir recursos elevados: deben ejecutarse en workers aislados y no dentro de la petición HTTP.
- El progreso no debe depender exclusivamente del socket: persistir estado y métricas permite recuperar la sesión.
- RNN, CNN y GAN tienen requisitos y objetivos distintos; el constructor debe validar opciones por familia.
- El uso de datasets exige definir formatos admitidos, licencias, privacidad, tamaño máximo y procedimiento de borrado.
- La infraestructura GPU puede elevar costos; iniciar con CPU y habilitar GPU mediante configuración y cuotas.
- No afirmar “tiempo real” solo por refrescar periódicamente: documentar el canal de eventos y el comportamiento de reconexión.

17. Entregables que debe producir el equipo

- Repositorio con frontend React/TypeScript, API Python y worker de entrenamiento.
- Documento de arquitectura, diagrama de componentes, modelo de datos y contratos API.
- Migraciones de base de datos, configuración de entorno de ejemplo y datos demo no sensibles.
- Suite de pruebas automatizadas y guía para ejecución local.
- Manual de usuario por módulo y guía operativa para despliegue, monitoreo y respaldo.
- Demostración reproducible de una red Feedforward entrenada con TensorFlow/Keras y seguimiento en tiempo real.

18. Criterio de finalización del MVP

El MVP se considera demostrable cuando un usuario autenticado crea un proyecto, configura una red Feedforward con capas y funciones de activación válidas, selecciona un dataset de prueba, inicia un entrenamiento real en TensorFlow/Keras, visualiza el estado y las métricas sin recargar la pantalla, consulta el resultado posteriormente y conserva la versión del modelo con trazabilidad. Las familias RNN, CNN y GAN se incorporan como opciones documentadas y se habilitan progresivamente según sus validaciones y pruebas.

19. Referencia curricular

Documento base: PIAD-427_CONTENIDO.pdf, “Machine Learning y Deep Learning”, Ingeniería de Software con Inteligencia Artificial, Semana 11. El documento fuente establece la tarea, las operaciones y los conocimientos tecnológicos que fundamentan este alcance. La arquitectura enterprise, módulos, entidades, endpoints y criterios de producción incluidos en este documento son una propuesta de diseño para convertir ese contenido en un sistema real.

Fin del documento.